

Message-Oriented Communication

From Sockets to Cloud-Native Event Buses

SE 424: Distributed Systems

Distributed Communication

Layered
Communication

RPC & RMI

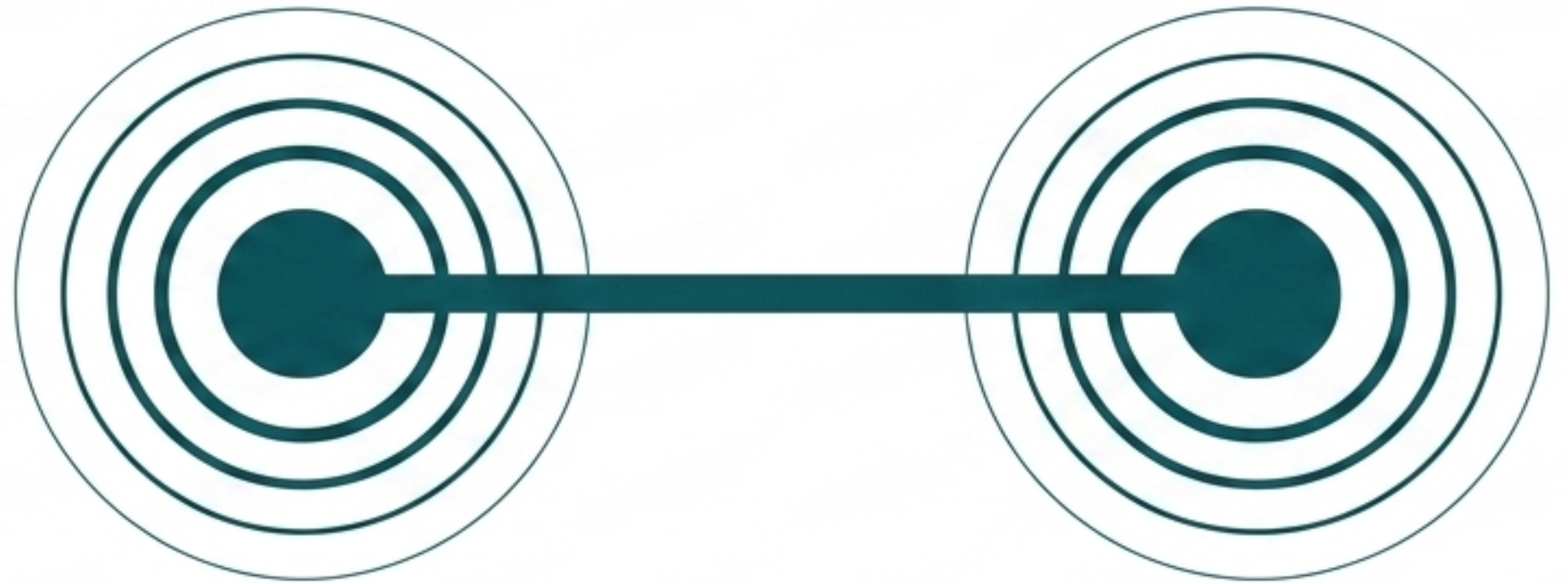
● **Message-Oriented
Communication**

Moving beyond remote procedure calls to independent messaging.

The Socket Interface

- Transient messaging paradigm
- Adopted by all POSIX systems
- Standard Berkeley socket interface

Endpoints must be active simultaneously.



Socket Operations

Socket

Create endpoint

Bind

Attach local address

Listen

Announce willingness

Accept

Block until request

Connect

Attempt connection

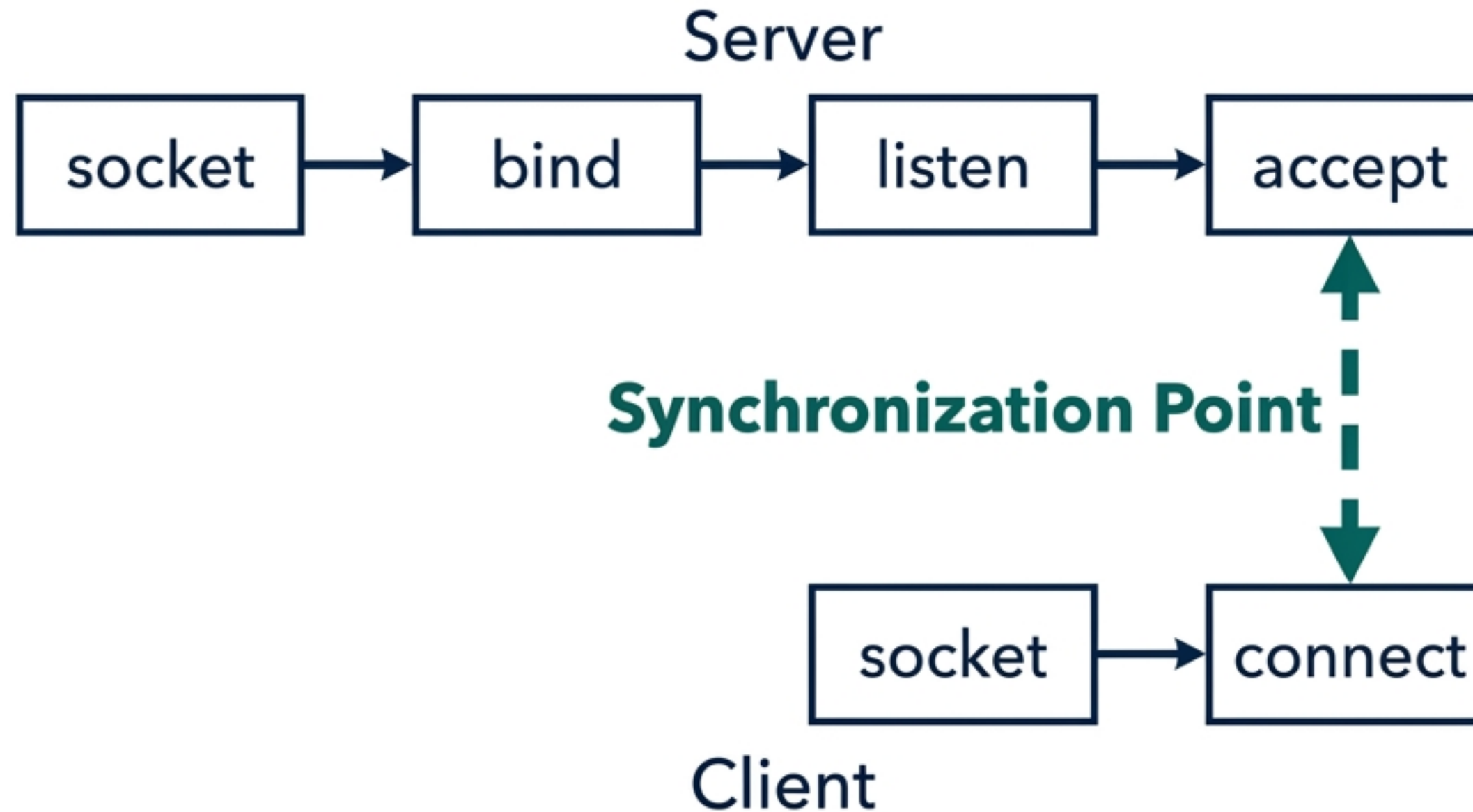
Send / Receive

Transfer data

Close

Release connection

Client-Server Blocking



Execution halts until both parties connect.

Message-Oriented Middleware

- Asynchronous communication
 - Persistent storage
 - Middleware-level queues

Sender and receiver no longer need to overlap in time.

Java Queue Primitives

Put

Append to queue

Get

Block and remove first

Poll

Remove first (Never block)

Notify

Trigger handler on arrival

Communication Paradigms

Sockets

MOM

Transient
Synchronous
Tightly Coupled

Persistent
Asynchronous
Loosely Coupled

Queues introduce buffers, enabling temporal decoupling.

Message Brokers

- Applications use differing formats
- Broker acts as an integration bus
- Transforms incoming messages



IBM MQ Architecture

- Regime of Queue Managers
- Unidirectional message channels
- Message Channel Agents (MCAs) wrap/unwrap packets

Industrial-grade, subject-based routing capability.

Channel Agents in Action



MCAs handle the lower-level TCP/IP complexity automatically.

The Modern Event Bus

- Decoupled backend microservices
- Publish/Subscribe channels
- Eventual consistency



Industry Implementations

Azure Event Hubs	Apache Camel	Amazon SQS
Real-time data ingestion	Open-source integration framework	Fully managed microservice queues

The underlying broker architecture is now a foundational cloud utility.

Key Takeaways

- Sockets require synchronous presence.
- Middleware queues provide asynchronous persistence.
- Brokers solve application heterogeneity.
- Event buses drive modern microservices.

End of Chapter 2.3

Distributed Systems